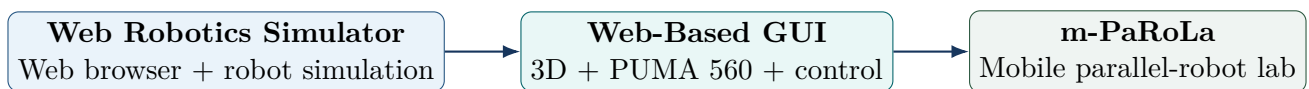


Three Robotics Research Papers

Explained in Very Simple English

A clear study guide for understanding web-based robotics simulation, robot kinematics, virtual laboratories, and mobile learning.



Main idea of all three papers

Make robotics learning more accessible by moving simulation into a web browser, while keeping the mathematics of robot kinematics visible and interactive.

Prepared as a learning and revision guide
based on the three papers in the **Research papers** folder.

Contents

1	How to Read These Three Papers	3
2	Paper 1 — Design and Development of a Web-Based Robotics Simulator	3
2.1	What is this paper about?	3
2.2	What does the simulator try to provide?	3
2.3	How is the system built?	3
2.4	Important technologies	4
2.5	Forward and inverse kinematics in this paper	4
2.6	Accuracy and limitations	4
2.7	What is the real contribution?	5
3	Paper 2 — A Web-Based Integrated GUI for 3D Modeling, Kinematic Study, and Control of Robotic Manipulators	6
3.1	What is this paper about?	6
3.2	Why did the authors choose a web-based approach?	6
3.3	WebGL and Three.js	6
3.4	How can a robot model be created?	6
3.4.1	Approach A — Build from basic shapes	6
3.4.2	Approach B — Import CAD geometry	7
3.5	The PUMA 560 and Denavit–Hartenberg parameters	7
3.6	Forward kinematics: what happens in the GUI?	7
3.7	Workspace analysis	7
3.8	Inverse kinematics: why is it harder?	8
3.9	Remote control of the real PUMA 560	8
3.10	Advantages and limitations	8
4	Paper 3 — m-PaRoLa: A Mobile Virtual Laboratory for the Kinematics of Parallel Robots	10
4.1	What is this paper about?	10
4.2	Why parallel robots?	10
4.3	The five-bar robot	10
4.3.1	Forward kinematics	10
4.3.2	Inverse kinematics	10
4.3.3	Workspace	11
4.3.4	Singularity	11
4.4	Changing the robot design	11
4.5	The 3RRR robot	11
4.6	3RRR forward and inverse kinematics	11
4.7	Workspace of the 3RRR robot	12
4.8	Why the mobile interface is special	12
4.9	Technology and evolution	12
5	The Three Papers Compared	13
6	The Most Important Concepts Across All Three Papers	13
6.1	1. Forward vs. Inverse Kinematics	13
6.2	2. Workspace	13
6.3	3. Multiple IK solutions	13
6.4	4. Singularities	13
6.5	5. Why web-based simulation matters	14
7	How These Papers Relate to KyntriX	15

8 Important Points to Remember — Paper by Paper	15
9 Quick Viva / Exam Questions	16
10 Final Mental Model	16

1 How to Read These Three Papers

These papers are easier to understand if you see them as three related steps.

1. **Paper 1:** asks, “Can robot simulation be made simple enough to run directly in a browser?”
2. **Paper 2:** goes deeper and asks, “Can a browser-based system provide a complete 3D robotics GUI, including PUMA 560 kinematics and even control of a real robot?”
3. **Paper 3:** focuses on mobile learning and asks, “Can students study the kinematics of parallel robots using a simulator that works well on phones and tablets?”

One-line connection: all three papers reduce dependence on heavy, installed robotics software and use interactive visual simulation to make robotics easier to learn.

2 Paper 1 — Design and Development of a Web-Based Robotics Simulator

2.1 What is this paper about?

The first paper presents a robotics simulator designed to run **directly inside a web browser**. The central motivation is simple: traditional robotics simulation software can require installation, dependencies, powerful hardware, or technical setup. The authors wanted a system where a user can open a browser and start learning and testing robot kinematics.

The software is developed mainly with JavaScript. It uses web technologies and open-source libraries for 3D graphics, kinematics, and the user interface.

Core problem: robotics simulation is useful, but many tools are difficult to install or require significant computing resources.

Core solution: move the simulation into the browser so that the user needs only a compatible web browser.

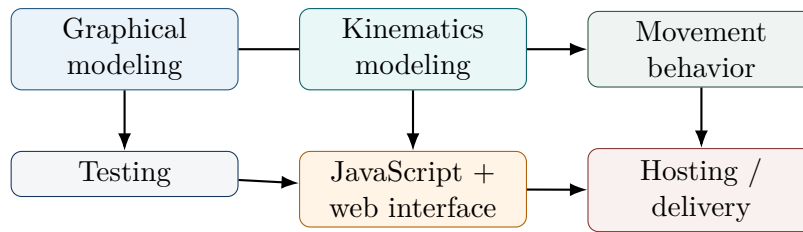
2.2 What does the simulator try to provide?

The paper lists several functions:

- Define a robot using parameters such as joint type, number of joints, link type, number of links, and link length.
- Move joints using translational or rotational motion.
- Represent orientation using Euler-angle style model-space information.
- Perform **forward kinematics** and **inverse kinematics**.
- Perform basic path planning.
- Display point positions and joint angles in real time.
- Support online simulation and code verification for supported real robots.
- Import and export robot models or code.

2.3 How is the system built?

The paper divides the development into major parts:



2.4 Important technologies

Three.js A JavaScript library used to create and display interactive 3D graphics. It uses WebGL underneath.

WebGL A browser technology for rendering interactive 2D and 3D graphics without requiring a traditional browser plugin.

HTML DOM

Provides the structure for the web interface.

Bootstrap Used for common interface styling and layout.

JavaScript

Implements the main simulation and kinematic behavior.

Very simple picture: Three.js/WebGL draws the robot. JavaScript performs the robot mathematics. HTML and Bootstrap provide the controls and page layout.

2.5 Forward and inverse kinematics in this paper

The simulator implements general forward and inverse kinematic models for robots with between **2 and 6 degrees of freedom**.

Forward kinematics (FK) starts with joint values and calculates the end-effector pose:

joint variables $\longrightarrow$ position + orientation.

For example, if the joint angles are known, the simulator can calculate where the end-effector is.

Inverse kinematics (IK) goes in the opposite direction:

desired position + orientation $\longrightarrow$ joint variables.

IK is generally harder because the equations are nonlinear and a target may have multiple solutions or no physically valid solution.

The paper uses the Denavit–Hartenberg framework for the robot geometry and implements the corresponding kinematic equations.

2.6 Accuracy and limitations

The paper reports a point-space mapping accuracy of about **0.05 cm per point-to-point map**. It also states that the system supports up to **6 DOF**.

The paper identifies important limitations:

- Maximum supported configuration is six degrees of freedom.
- Point-space mapping has a stated accuracy limit.
- Real-robot online simulation is limited to supported robots, including models using Karel programming language such as some FANUC and KUKA robots.

- Interoperability with real machines still needs further verification and improvement.

2.7 What is the real contribution?

The most important contribution is not a new robotics equation. It is the **delivery method**: the authors demonstrate that useful robot kinematic simulation can be delivered through a browser without heavy, resource-intensive desktop software.

Remember Paper 1

Browser-based robotics simulator; JavaScript; Three.js/WebGL; FK and IK; 2–6 DOF; path planning; real-time joint information; import/export; lightweight and platform-independent idea; real-machine interoperability remains limited.

3 Paper 2 — A Web-Based Integrated GUI for 3D Modeling, Kinematic Study, and Control of Robotic Manipulators

3.1 What is this paper about?

The second paper is more detailed and more complete. It presents an integrated web-based graphical interface for:

1. creating or importing a 3D robot model,
2. studying robot kinematics,
3. visualizing motion,
4. analyzing workspace,
5. examining multiple inverse-kinematic solutions, and
6. controlling a real PUMA 560 robot through the interface.

The main robot used for demonstration is the **PUMA 560**, a six-degree-of-freedom serial industrial manipulator.

Main idea: combine the mathematics, the 3D robot, the graphical interface, and real-robot control in one web-based environment.

3.2 Why did the authors choose a web-based approach?

Traditional tools such as MATLAB-based environments and other robotics simulators can be powerful, but they may depend on installed software, plugins, hardware, or specific system configurations.

The paper argues that a browser-based environment can improve accessibility for education and virtual laboratories. Students can access the experiment from different devices without installing the complete simulation package.

3.3 WebGL and Three.js

WebGL is the browser-level technology that allows hardware-accelerated 2D and 3D graphics. However, directly programming complex 3D models with raw WebGL can be cumbersome.

Therefore, the authors use **Three.js**, a higher-level JavaScript library. Three.js simplifies:

- 3D object creation,
- camera control,
- lighting,
- animation,
- scene management, and
- interactive visualization.

3.4 How can a robot model be created?

The paper describes two broad approaches.

3.4.1 Approach A — Build from basic shapes

Simple geometries can be combined to form the robot. A 3D component is defined by its geometry and material; together they form a mesh, which is then placed in the scene.

This is useful when robot links can be represented reasonably well by basic geometric shapes.

3.4.2 Approach B — Import CAD geometry

For more complex robots, a CAD model can be prepared using tools such as SolidWorks, CATIA, Autodesk, or Creo. The geometry can then be converted/imported into a web-compatible representation.

Simple distinction:

Basic shapes = easier to build programmatically.

CAD import = better for detailed, realistic robot geometry.

3.5 The PUMA 560 and Denavit–Hartenberg parameters

The PUMA 560 is a 6-DOF serial robot with six rotary joints. The paper uses a modified Denavit–Hartenberg convention.

The four standard D–H quantities are:

$$\begin{aligned} a_i &= \text{link length}, & \alpha_i &= \text{link twist}, \\ d_i &= \text{link offset}, & \theta_i &= \text{joint angle}. \end{aligned}$$

A homogeneous transformation matrix describes the relationship between neighboring coordinate frames. The overall end-effector transformation is obtained by multiplying the six transformations:

$${}^0T_6 = {}^0T_1 {}^1T_2 {}^2T_3 {}^3T_4 {}^4T_5 {}^5T_6.$$

The resulting matrix has the familiar structure

$${}^0T_6 = \begin{bmatrix} r_{11} & r_{12} & r_{13} & P_x \\ r_{21} & r_{22} & r_{23} & P_y \\ r_{31} & r_{32} & r_{33} & P_z \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

The last column gives the end-effector position, while the upper-left 3×3 block represents orientation.

3.6 Forward kinematics: what happens in the GUI?

The user enters joint angles. The interface then does two things at the same time:

1. moves the 3D PUMA robot to the requested configuration;
2. calculates and displays the transformation matrix and end-effector position/orientation.

The interface can also move individual joints through slider controls.

3.7 Workspace analysis

The **reachable workspace** is the total region in space that the end-effector can reach within the allowed joint ranges.

The paper visualizes the workspace by plotting end-effector positions. A point plot shows sampled positions; a line plot can be used to visualize a motion trajectory.

This is useful because it connects the mathematics to a visible geometric result.

3.8 Inverse kinematics: why is it harder?

For FK, the joint angles are known, so the transformation matrices can be multiplied directly.

For IK, the desired position and orientation are known, and we must find the joint angles. Because the equations are nonlinear, there may be:

- no solution,
- one solution, or
- multiple solutions.

For the PUMA 560, the paper notes that a reachable target can have up to **eight joint solutions**, although joint limits can remove some of them.

The GUI calculates the possible solutions and lets the user click through them to see the corresponding robot configurations.

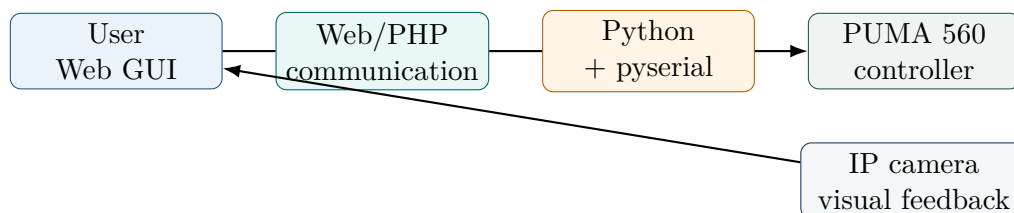
Important: mathematical IK solutions are not automatically the same as physically usable robot configurations. Joint limits and other practical constraints can eliminate solutions.

3.9 Remote control of the real PUMA 560

This is one of the most important parts of Paper 2.

The system is not limited to simulation. The web interface sends joint commands through a communication chain involving PHP and Python. Python uses the `pyserial` module to communicate with the robot controller.

An IP camera provides visual feedback.



This creates a useful learning loop: **command the robot virtually, send the command to the real robot, and observe the real movement.**

3.10 Advantages and limitations

Advantages:

- Browser-based access.
- No traditional browser plugin is required.
- 3D visualization makes mathematical concepts easier to see.
- FK, IK, workspace and trajectory concepts are combined.
- Multiple IK solutions can be visualized.
- Real PUMA 560 control is demonstrated.
- Virtual and real robot views can be compared.

Limitations/future work described by the authors:

- More complete dynamic analysis could be added.

- Physics can be added to the scene.
- More robot programming support could be included.
- Broader application to other robotics areas is possible.

Remember Paper 2

PUMA 560; 6 DOF; modified D-H parameters; FK and IK; up to eight IK configurations for reachable targets; workspace plotting; Three.js/WebGL; CAD import; integrated GUI; real PUMA control using PHP/Python/serial communication; IP-camera feedback.

4 Paper 3 — m-PaRoLa: A Mobile Virtual Laboratory for the Kinematics of Parallel Robots

4.1 What is this paper about?

The third paper focuses on **mobile learning**. It presents m-PaRoLa, a mobile virtual laboratory for studying the kinematics of parallel robots.

The simulator can run through a web browser on desktop computers and mobile devices such as smartphones and tablets.

The paper concentrates on two parallel robots:

1. a **five-bar robot**, and
2. a **3RRR planar parallel robot**.

The main educational concepts are forward kinematics, inverse kinematics, workspace, singularities, and how robot geometry affects those properties.

Main idea: take a robotics virtual laboratory and redesign its interaction so that it remains understandable and usable on a small mobile screen.

4.2 Why parallel robots?

A serial robot has a chain of links from the base to the end-effector. A parallel robot has multiple kinematic chains working together.

Parallel robots can offer high acceleration and useful mechanical properties, but their kinematics can be more complicated. In particular, their workspace, singularities, and multiple solutions are important topics for students.

4.3 The five-bar robot

The five-bar robot is a planar parallel mechanism. Its end-effector point is connected to the fixed base through two kinematic chains.

The paper studies:

- forward kinematics,
- inverse kinematics,
- workspace,
- singularities, and
- how changing geometric dimensions changes the workspace.

4.3.1 Forward kinematics

Given the two input joint angles θ_1 and θ_2 , the simulator calculates the position (x, y) of the gripper point.

Importantly, the forward problem can have **two solutions**. The simulator lets the user switch between them.

4.3.2 Inverse kinematics

The user gives the desired gripper position (x, y) . The simulator calculates the required joint angles.

Each kinematic chain can have two solutions, so different combinations can produce multiple overall configurations.

4.3.3 Workspace

The workspace is the region where the gripper can physically reach.

The simulator draws the workspace boundary. If the user drags the gripper outside the reachable region, the robot appears to “break” because the inverse-kinematic equations no longer have real solutions for that requested position.

4.3.4 Singularity

A singularity is a configuration where the robot’s kinematic behavior changes in a special way and normal motion/solution properties can become problematic.

In the simulator, singular curves are shown graphically. The user can move the robot toward these curves and see how the kinematic chains stretch or fold.

4.4 Changing the robot design

One particularly useful feature is that the user can change geometric parameters interactively.

For example, dragging certain joints changes link lengths. The workspace and singular curves are then recomputed.

This teaches an important engineering idea:

$$\text{robot geometry} \longrightarrow \text{kinematic behavior} \longrightarrow \text{workspace and singularities.}$$

The user does not only study a fixed robot; the user can experiment with the design itself.

4.5 The 3RRR robot

The second mechanism in m-PaRoLa is the planar **3RRR parallel robot**. It has a triangular moving platform connected to a fixed base through three parallel kinematic chains.

It has three degrees of freedom:

$$(x, y, \phi),$$

where x and y describe translation of the platform center and ϕ describes its orientation.

The three controlled joint angles are:

$$\theta_1, \quad \theta_2, \quad \theta_3.$$

4.6 3RRR forward and inverse kinematics

For forward kinematics, the input is

$$(\theta_1, \theta_2, \theta_3)$$

and the output is

$$(x, y, \phi).$$

The paper explains that the forward kinematic problem has six solutions in the complex domain. Depending on the robot geometry and joint values, two, four, or six of those solutions may be real.

For inverse kinematics, the desired (x, y, ϕ) is given. Each of the three chains can have two possible joint solutions, leading to up to

$$2^3 = 8$$

inverse configurations.

Why does $2^3 = 8$ appear?

If joint 1 has two choices, joint 2 has two choices, and joint 3 has two choices, the possible combinations are $2 \times 2 \times 2 = 8$.

4.7 Workspace of the 3RRR robot

The complete workspace is a set in three-dimensional configuration space:

$$(x, y, \phi).$$

For easier visualization, the paper studies the **constant-orientation workspace**: the set of (x, y) positions that are reachable while ϕ is fixed.

The boundary is shown graphically. Changing the robot dimensions or changing ϕ changes the workspace.

4.8 Why the mobile interface is special

A desktop monitor has much more screen space than a phone. A phone also has more limited input methods.

Therefore, m-PaRoLa uses interaction elements with multiple purposes. The interface is deliberately minimalist, and graphical interaction is preferred over large menus and many windows.

For example, a draggable circle can be used to change a joint value. Another interaction can switch between kinematic solutions.

Educational lesson: good mobile simulation is not simply “the desktop simulator made smaller.” The interface itself has to be redesigned around the limitations of small screens and touch interaction.

4.9 Technology and evolution

m-PaRoLa evolved from the earlier PaRoLa virtual laboratory. The paper describes a JavaScript-based mobile virtual laboratory approach and discusses reusing existing kinematic calculations while adapting the user experience for mobile devices.

The future direction described includes adding more robots, including possible remote real robots, and using a client-server model in which a browser-side JavaScript simulator communicates with existing kinematic computation code.

Remember Paper 3

m-PaRoLa; mobile virtual laboratory; five-bar and 3RRR parallel robots; FK/IK; workspace; singularities; multiple solutions; interactive geometry changes; JavaScript; browser/mobile access; minimalist touch-friendly GUI.

5 The Three Papers Compared

	Paper 1	Paper 2	Paper 3
Main goal	Browser robotics simulator	Integrated web GUI + real PUMA control	Mobile virtual lab for parallel robots
Main robot focus	General robots, 2–6 DOF	PUMA 560, 6 DOF	Five-bar and 3RRR parallel robots
Main mathematics	FK, IK, D–H	D–H, FK, IK, workspace	FK, IK, workspace, singularities
3D visualization	Yes	Yes, detailed	Mainly 2D planar visualization
Web technology	JavaScript, Three.js, WebGL	HTML5, WebGL, Three.js, JavaScript	JavaScript/browser approach
Real robot	Limited supported models	PUMA 560 control demonstrated	Future remote-robot direction
Mobile focus	General browser accessibility	Browser accessibility	Strong mobile-first focus
Key educational idea	Remove installation barriers	Connect theory, simulation and real robot	Learn through touch and visual experimentation

6 The Most Important Concepts Across All Three Papers

6.1 1. Forward vs. Inverse Kinematics

This is the single most important robotics distinction.



- **FK:** “I know the joints. Where is the robot’s end-effector?”
- **IK:** “I know where I want the end-effector. What joint values can put it there?”

6.2 2. Workspace

Workspace means the region that the robot’s end-effector can reach.

Workspace = all physically reachable end-effector poses/positions

A target outside the workspace cannot be reached by that robot configuration and geometry.

6.3 3. Multiple IK solutions

Robots can sometimes reach the same target in several different postures.

This is why a good educational simulator should not show only one answer. Papers 2 and 3 emphasize visualizing multiple solutions.

6.4 4. Singularities

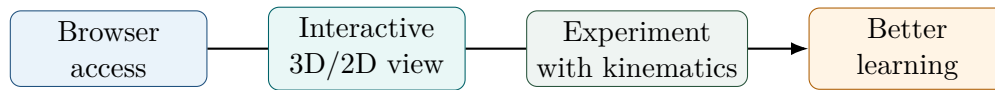
A singularity is a special robot configuration where the normal relationship between joint motion and end-effector motion becomes problematic. In parallel robots, singular curves can divide the workspace into different regions.

The key exam-level idea is:

Singularities are special configurations where the robot's kinematic behavior becomes degenerate or loses normal controllability properties.

6.5 5. Why web-based simulation matters

The three papers repeatedly connect simulation with accessibility.



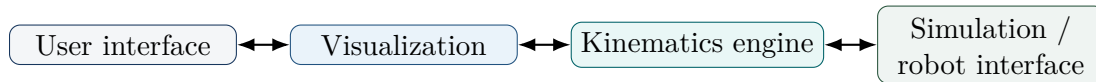
The papers are therefore not just about software engineering. They are also about **robotics education**.

7 How These Papers Relate to KyntriX

These papers are especially relevant when thinking about a robotics learning and simulation platform such as KyntriX.

- **Paper 1** supports the idea of a browser-first simulator that avoids heavy installation.
- **Paper 2** shows how a single interface can connect 3D modeling, kinematics, workspace visualization, and real-robot communication.
- **Paper 3** shows the educational value of interactive workspace and singularity visualization, especially for parallel robots and mobile users.

The common architectural idea can be summarized as:



This is a useful mental model for understanding why the papers are relevant to a modern robotics platform: the user should be able to change a parameter, immediately see the mathematical result, and immediately see the robot behavior.

8 Important Points to Remember — Paper by Paper

Paper 1: Web Robotics Simulator

1. Browser-based robotics simulation is the main contribution.
2. JavaScript is used for the simulation logic.
3. Three.js and WebGL provide interactive graphics.
4. The simulator supports FK and IK.
5. The stated supported range is 2–6 DOF.
6. Path planning and point/joint visualization are included.
7. Import/export and supported real-robot interaction are discussed.
8. The major limitation is real-machine interoperability and the six-DOF ceiling.

Paper 2: Web-Based GUI

1. PUMA 560 is the main example.
2. PUMA 560 has 6 rotary DOF.
3. Modified D–H parameters are used.
4. FK maps joint values to end-effector pose.
5. IK maps desired pose to possible joint solutions.
6. A reachable PUMA target can have up to eight configurations in the mathematical solution set.
7. Workspace can be plotted and trajectories can be visualized.
8. CAD import and basic-shape modeling are both possible.
9. The GUI can communicate with a real PUMA 560.
10. Python + pyserial + serial communication + IP-camera feedback connect simulation/control with the physical robot.

Paper 3: m-PaRoLa

1. It is a mobile virtual robotics laboratory.
2. It studies parallel robots.
3. The two main robots are five-bar and 3RRR.
4. It teaches FK, IK, workspace, and singularities.
5. Five-bar FK has two solutions.
6. 3RRR IK can have up to eight configurations because each of three chains can have two choices.
7. 3RRR FK can have multiple solutions in the complex domain, with some becoming real depending on the configuration.
8. Workspace and singularity curves can be changed by changing robot geometry.
9. The GUI is deliberately minimalist for small screens and limited input methods.

9 Quick Viva / Exam Questions

What is FK? Given joint variables, calculate the end-effector pose.

What is IK? Given the desired end-effector pose, calculate possible joint variables.

What is workspace?
The region of end-effector positions or poses that the robot can reach.

Why can IK have multiple answers?
Because nonlinear robot geometry can permit several valid configurations for the same target.

Why use D–H parameters?
They provide a systematic way to describe robot link geometry and coordinate-frame transformations.

Why use WebGL?
It enables interactive 2D/3D graphics directly in compatible browsers.

Why use Three.js?
It makes complex WebGL-based 3D scene creation easier.

What is special about Paper 2?
It connects web simulation with control and visual feedback from a real PUMA 560.

What is special about Paper 3?
It adapts robotics virtual-lab interaction for smartphones and tablets and focuses on parallel robots.

Why are singularities important?
They are special configurations where normal kinematic behavior becomes degenerate and can separate regions of valid motion/solutions.

10 Final Mental Model

If you remember only one story, remember this:

Paper 1: Put robotics simulation in the browser.

Paper 2: Build a complete web robotics GUI around PUMA 560 and connect it to a real robot.

Paper 3: Make virtual robotics learning work well on mobile devices and use parallel robots to teach FK, IK, workspace and singularities.

The common educational cycle is:

Choose robot → change parameters → solve kinematics → visualize → experiment → understand

Bottom line: The three papers show different implementations of the same broad educational philosophy: robotics becomes easier to learn when the mathematics is connected directly to an interactive visual model that students can access without unnecessary software barriers.

Source Papers

1. “Design and Development of a Web-Based Robotics Simulator,” Dr. Laxmi Narayan Pattanaik and Amit Kumar Nandi.
2. “A web-based integrated GUI for 3D modeling, kinematic study, and control of robotic manipulators,” Ujjal Dey and Kumar Cheruvu S., *Computer Applications in Engineering Education*, 2020.
3. “m-PaRoLa: a mobile virtual laboratory for the kinematics of parallel robots,” Adrián Peidró et al., *IFAC-PapersOnLine*, 2018.